

UniversidadeVigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria do Traballo de Fin de Grao que presenta

D. Nome Alumna/o

para a obtención do Título de Graduado en Enxeñaría Informática

Título do Traballo de Fin de Grado



Xullo, 20XX

Traballo de Fin de Grao N°:

Titor/a:

Área de coñecemento: Linguaxes e Sistemas Informáticos

Departamento: Informática

Agradecimientos

[Texto]

Dedicatoria

[Texto]

Índice

1. Introducción	5
2. Objetivos	6
2.1. Objetivo principal	6
2.2. Objetivos específicos	6
2.3. Alcance y limitaciones	6
3. Antecedentes y contexto	7
3.1. Deep Reinforcement Learning en Ajedrez	7
3.2. Transformers en ajedrez	7
4. Resumen de la solución propuesta	9
5. Marco teórico y práctico. Herramientas empleadas	10
5.1. Marco teórico	10
5.1.1. Diseño Experimental e Hipótesis	10
5.1.2. Entorno: Reglas del ajedrez 5x5	10
5.1.3. Arquitectura basada en Transformers: Encoder y Self-Attention	13
5.1.4. Modelado del Aprendizaje por Refuerzo	13
5.2. Marco práctico	14
5.2.1. Codificación del Espacio de Estados (Input)	14
5.2.2. Codificación del Espacio de Acciones (Output)	15
5.2.3. Arquitectura Transformer	16
5.2.4. Algoritmo de Aprendizaje por Refuerzo: Proximal Policy Optimization (PPO)	18
5.2.5. Herramientas y tecnologías empleadas	20
6. Planificación y seguimiento	21
7. Principales aportaciones	22
7.1. Resultados generales en el entorno de Minichess	22
7.2. Efectos de la representación sobre la eficiencia y calidad del aprendizaje	22
7.3. Efectos de la inicialización supervisada sobre la convergencia del aprendizaje por refuerzo	22
8. Conclusiones	22
9. Vías de trabajo futuro	23
Referencias	24
A. Anexos	26
A.1. Hiperparámetros del Entrenamiento	26

Índice de figuras

1.	Posición inicial del ajedrez Gardner.	11
2.	Visualización de codificaciones de entrada. De izquierda a derecha: a, b, c... . . .	15
3.	Visualización de codificaciones de salida. De izquierda a derecha: a, b, c... . . .	16
4.	[Placeholder] Diagrama de la arquitectura utilizada (generado con IA, tiene errores / inconsistencias).	18
5.	Evolución de la pérdida según las distintas codificaciones del tablero estudiadas.	22
6.	Tasa de victoria y avance del ELO estimado a lo largo del entrenamiento. . . .	23

Índice de cadros

1.	Resultados de rendimiento final comparando representaciones (Hipótesis 2). . .	22
2.	Comparativa de convergencia pre-entrenamiento vs tabula rasa (Hipótesis 3). .	23
3.	Tabla de hiperparámetros de los modelos y del algoritmo PPO.	26

1. Introducción

“Chess is a game of luck.
If you have a good opponent you have
bad luck, and if you have a bad
opponent you have good luck.”^[1]

Jacob Segal, 7 años

Este Trabajo de Fin de Grado consiste en la aplicación y estudio del Aprendizaje por Refuerzo (RL) sobre una variante de ajedrez simplificada, el ajedrez 5×5 (ajedrez de Gardner o Minichess). El objetivo central es validar el uso de arquitecturas Transformer en tareas de RL, estudiando el efecto de las representaciones del tablero y comprobando si el preentrenamiento supervisado mejora la convergencia del agente.

Dado que el ajedrez 8×8 demanda una capacidad computacional y un tiempo de ejecución que excede con creces los recursos disponibles en el marco del trabajo, se ha optado por utilizar el ajedrez 5×5 . Este entorno reducido mantiene las dinámicas esenciales del aprendizaje por refuerzo adversarial, como las acciones legales, las recompensas diferidas y la planificación estratégica, pero se mantiene computacionalmente manejable para la experimentación ágil.

El resto de este documento se estructura de la siguiente manera: en la Sección 2 se detallan los objetivos generales y específicos junto con el alcance del trabajo; en la Sección 3 se describe el estado del arte en transformers, aprendizaje por refuerzo y ajedrez y los antecedentes del problema; en la Sección 4 se introduce y resume la solución metodológica planteada.

En la Sección 5 se desarrolla el marco teórico (5.1) y práctico (5.2), detallando las hipótesis a probar, el modelado matemático del entorno de ajedrez, de la arquitectura transformer, la codificación de entradas y salidas y del problema de RL, además de justificar las herramientas utilizadas para el desarrollo. En la Sección 6 se muestra la planificación temporal del proyecto.

Las secciones finales presentan las aportaciones principales y resultados empíricos de los experimentos (Sección 7), las conclusiones y discusión entorno a los resultados (Sección 8) y las futuras líneas de investigación y limitaciones del trabajo (Sección 9). Los anexos contienen información adicional específica, o desarrollos / cuestiones que van mas allá del alcance del trabajo (Sección A).

2. Objetivos

2.1. Objetivo principal

Diseñar, entrenar y evaluar un agente de Inteligencia Artificial basado en arquitecturas Transformer mediante Aprendizaje por Refuerzo para el entorno de ajedrez 5×5 .

2.2. Objetivos específicos

1. Implementar un algoritmo de RL para ajedrez 5×5 , junto a la lógica necesaria para self-play.
2. Desarrollar y adaptar una arquitectura Transformer para procesar estados del tablero y predecir distribuciones de probabilidad sobre el espacio de acciones.
3. Evaluar empíricamente el impacto de distintas representaciones del tablero y del espacio de acciones en la velocidad de aprendizaje y rendimiento del modelo.
4. Comparar el rendimiento y la eficiencia de convergencia entre un agente entrenado mediante RL *tabula rasa* frente a un agente con inicialización mediante aprendizaje supervisado.

2.3. Alcance y limitaciones

El alcance de este trabajo se ha enmarcado en los siguientes límites prácticos e investigativos:

- **Entorno de juego:** Se limita estrictamente a la variante Gardner de ajedrez 5×5 . No se explorarán otras variantes del miniajedrez ni el ajedrez estándar 8×8 .
- **Recursos computacionales:** El entrenamiento de los modelos se realizará utilizando hardware de consumo accesible (una GPU de gama media/alta) con un límite temporal de entrenamiento acotado a un número de horas que permita la experimentación iterativa ágil.
- **Algoritmos de aprendizaje:** Se prioriza el uso de métodos de optimización basados en gradiente de política (PPO), evaluando el *Model-Free RL*. No se hará uso de algoritmos basados en planificación o búsqueda explícita (*Lookahead Search* como MCTS), con el fin de optimizar el tiempo de ejecución en hardware de consumo.

3. Antecedentes y contexto

El Aprendizaje por Refuerzo (RL) ha demostrado en los últimos años resultados extraordinarios en numerosos campos como el Go, el ajedrez, la síntesis de proteínas, los modelos de lenguaje y la robótica. Con la enorme capacidad de computación actual, que sigue creciendo día a día, el RL aprovecha las características enunciadas en *The Bitter Lesson* de Sutton^[2]: la búsqueda masiva y el aprendizaje de funciones a través del muestreo superan a largo plazo a los enfoques basados en conocimiento humano inyectado manualmente.

Ambos procesos, búsqueda y aprendizaje, son altamente exigentes en términos de computación, pero bajo la suposición de que esta aumenta al ritmo de la Ley de Moore, los métodos que aprovechan la computación superan eventualmente aquellos basados en conocimiento humano. Esto es especialmente cierto en entornos (discretos o continuos) con espacios de estado masivos, como son el ajedrez o la robótica.

3.1. Deep Reinforcement Learning en Ajedrez

Desde la irrupción del aprendizaje profundo y el aprendizaje por refuerzo en los juegos de tablero, se han explorado diversas arquitecturas neuronales. El hito inicial lo marcó AlphaGo^[3], que combinó redes convolucionales (CNNs) con búsqueda de árbol Monte Carlo (MCTS) para vencer al campeón mundial de Go. Posteriormente, esta metodología se generalizó con AlphaZero^[4], un algoritmo de RL capaz de dominar el ajedrez, el shogi y el Go partiendo desde cero (*tabula rasa*) y utilizando una única red neuronal para evaluar posiciones y priorizar jugadas. El impacto de estos métodos fué más allá, al demostrar el poder del *deep reinforcement learning* en otros dominios científicos complejos, como la predicción de estructuras de proteínas mediante AlphaFold^[5].

Paralelamente, en el ámbito de los motores de ajedrez tradicionales, enfoques como NNUE (*Efficiently Updatable Neural Networks*)^[6] tomaron un camino alternativo: en lugar de grandes redes convolucionales, emplean arquitecturas *feed-forward* muy ligeras con sesgos inductivos específicos del ajedrez, optimizadas para ejecutarse en CPU y priorizar la velocidad de evaluación y la profundidad de búsqueda táctica.

En estos motores de juego, las redes neuronales actúan como la parte *estratégica* mientras que la búsqueda actúa como la parte *táctica*, decidiendo a corto plazo bajo una serie de restricciones más claras (jugadas forzadas, amenazas claras como jaques o pérdida de material), y depende más del *cálculo*. Esto es, en cierto modo, reminiscente de cómo los humanos estructuramos nuestro pensamiento a la hora de jugar al ajedrez.

3.2. Transformers en ajedrez

Entre los enfoques anteriores, ninguno hacía uso de mecanismos de atención ni de arquitecturas basadas en Transformers, lo que nos lleva a explorar el potencial de estos últimos. Más recientemente, tras el éxito de esta arquitectura en los modelos de lenguaje (LLMs), han surgido aproximaciones que buscan aplicarla a tareas de RL y ajedrez. En este trabajo, se busca demostrar que la arquitectura Transformer es altamente competitiva para la tarea de RL, utilizando un entorno 5×5 para facilitar la simulación y el muestreo iterativo.

En este ámbito, Ruoss et al.^[7] demostraron la viabilidad de realizar planificación amortizada utilizando Transformers a gran escala aplicados al ajedrez. Por otro lado, Monroe y Chalmers^[8] desarrollaron un modelo basado en Transformers para dominar el ajedrez, sentando las bases para Chessformer, una arquitectura unificada propuesta posteriormente por Monroe et al.^[9]. Además, Jenner et al.^[10] aportaron evidencia sobre la existencia de capacidades de anticipación y planificación (*look-ahead*) aprendidas de forma implícita por este tipo de redes, mientras que el proyecto open source Leela Chess Zero también ha incorporado y evaluado estas arquitecturas en su motor de juego^[11].

Por su parte, el estudio del miniajedrez o ajedrez en tableros reducidos tiene una larga trayectoria. Existen miles de variantes del ajedrez; la variante de Gardner fue propuesta originalmente por Martin Gardner en 1969^[12]. Este tipo de entornos se ha empleado tradicionalmente para analizar la complejidad matemática del juego y estudiar algoritmos de búsqueda heurística, al reducir drásticamente el espacio de estados sin perder la naturaleza del ajedrez convencional; necesidad de táctica y estrategia, y emergencia de dinámicas interesantes al contar con distintos tipos de pieza (lo que aumenta el número de combinaciones posibles). El ajedrez 5×5 es particularmente interesante ya que permite mantener todos los tipos de pieza (torre, alfil, caballo, dama y rey) y una disposición inicial similar a la del ajedrez estándar.

En 2013, Mehdi y Prost resolvieron débilmente el ajedrez Gardner, esto es, obtuvieron un algoritmo que permite asegurar el resultado de la partida partiendo de la posición inicial, pero no desde cualquier posición arbitraria. Utilizando técnicas de búsqueda e intervención humana, obtuvieron un oráculo para el jugador blanco y negro que asegura un empate partiendo de la jugada inicial.^[13]

4. Resumen de la solución propuesta

[TODO: Redactar bien el pipeline, por ahora solo es un esqueleto. Como es un resumen, tampoco tiene que ser super extenso, un par de líneas por cada parte quizás. Btw, no se solapa algo con la planificación?]

1. Generación de un dataset de partidas de Minichess con un motor clásico.
2. Estudio, filtrado y procesado de datos.
3. Diseño de arquitectura de un modelo Transformer Encoder.
4. Evaluación de distintos sesgos de input / output en fase de entrenamiento supervisado.
5. Definición del bucle de entrenamiento e implementación de algoritmo de RL (PPO).
6. Fase de evaluación contra una *baseline* (ej. Stockfish o heurísticas simples).
7. Evaluación de red preentrenada VS. from-scratch (mismo numero de epochs).

5. Marco teórico y práctico. Herramientas empleadas

5.1. Marco teórico

5.1.1. Diseño Experimental e Hipótesis

Para dar respuesta a los objetivos planteados, el desarrollo práctico se articula en torno a la demostración de las siguientes hipótesis experimentales, las cuales estructuran el diseño experimental del trabajo:

- **Hipótesis 1 (Reducción del problema):** El ajedrez 5x5 como entorno simplificado de RL puede conservar suficientes dinámicas esenciales del aprendizaje por refuerzo adversarial al tiempo que se mantiene computacionalmente manejable para la experimentación.
- **Hipótesis 2 (Representación):** El uso de representaciones estructuradas del tablero y del espacio de acciones (ej. planos espaciales vs. predicción por casillas origen/destino) mejora la eficiencia del aprendizaje frente a representaciones planas.
- **Hipótesis 3 (Inicialización):** Un agente inicializado mediante aprendizaje supervisado a partir de jugadas generadas por un motor de referencia alcanza un nivel de rendimiento competitivo en menos *epochs* que un agente entrenado de forma puramente *tabula rasa*.

5.1.2. Entorno: Reglas del ajedrez 5x5

El ajedrez 5×5 en su variante de Gardner es una reducción matemática del ajedrez estándar. Las reglas básicas de movimiento y captura para cada pieza individual son idénticas a las del ajedrez estándar 8×8 , con las siguientes modificaciones estructurales:

- **Tablero:** tamaño de 5×5 casillas (columnas *a* a *e*, filas 1 a 5).
- **Disposición inicial:** La misma que el ajedrez estándar, pero con las piezas del lado de rey eliminadas, y una sola fila de separación entre peones. Cada jugador cuenta con 10 piezas. En la fila 1 (blancas) y fila 5 (negras) se colocan las piezas mayores en el orden: Torre (*R*), Caballo (*N*), Alfil (*B*), Dama (*Q*) y Rey (*K*). Los peones correspondientes se colocan en la fila 2 (blancas) y fila 4 (negras).
- **Omisión de reglas especiales:** No existe el enroque ni el avance doble de peón. Como consecuencia directa, tampoco existe la captura al paso (*en passant*).
- **Promoción de peones:** Cuando un peón alcanza la fila límite opuesta (fila 5 para blancas, fila 1 para negras), es obligatorio promocionarlo a una de las siguientes cuatro piezas: Dama, Torre, Alfil o Caballo.
- **Condiciones de fin de partida:** La partida finaliza por jaque mate, ahogado (*stalemate*), insuficiencia de material para dar jaque mate, si pasan 50 movimientos sin capturas ni avances de peón, o tablas por triple repetición (tres posiciones iguales consecutivas en el tablero).

Esta variante mantiene la naturaleza estratégica y la profundidad del ajedrez, pero en una fracción de su complejidad de cálculo original.

Representación de estados mediante notación FEN

Para interactuar con el entorno de ajedrez y alimentar los modelos de aprendizaje, se utiliza la notación FEN (*Forsyth-Edwards Notation*). Esta notación describe una posición particular del tablero mediante una cadena de texto que codifica la ubicación de las piezas, el jugador al que le toca mover y las disponibilidades de reglas especiales. En el contexto del ajedrez 5x5, el FEN se simplifica ya que cierta información no es aplicable a la variante de Gardner (como los enroques o las capturas *en passant*).

El FEN inicial de la variante Gardner se representa como:

`rnbqk/ppppp/5/PPPPP/RNBQK w - - 0 1`

La Figura 1 ilustra la disposición física y la colocación de las piezas en el tablero Gardner al inicio de la partida.



Figura 1: Posición inicial del ajedrez Gardner.

El ajedrez como juego y proceso teórico de decisión

El ajedrez es un juego de información perfecta y completa. En todo momento los jugadores tienen información total acerca de las funciones de utilidad de su rival y del estado de la partida, tanto presente como pasado. Esto proporciona fundamentos teóricos sólidos para probar algoritmos de RL; un entorno determinista y con funciones de transición bien definidas.

Dado el factor de ramificación estimado del ajedrez estándar (media de 35 hijos por nodo) y la complejidad del árbol de juego (10^{120}), propuesta por Claude Shannon en su artículo fundacional^[14], la variante de ajedrez 5×5 se establece como un entorno matemáticamente equivalente en sus reglas de transición, pero con un espacio de estados y una complejidad del árbol de juego acotados.

Para estimar la complejidad de la variante Gardner 5×5 , siguiendo un razonamiento similar al de Shannon, se pueden calcular las siguientes métricas:

- **Espacio de estados $\mathcal{S}$:** Mehdi y Prost estimaron que el número total de posiciones legales en el ajedrez 5×5 es aproximadamente 9×10^{18} ^[13], descontando posiciones ilegales (reyes en jaque simultáneo, peones en las filas de inicio/promoción, etc...).

TODO: no mencionan como lo obtuvieron. puede que sea un recuento combinatorio estilo $\text{total} = \text{permutaciones}(25, 10) * \text{comb}(15, 5) * \text{comb}(10, 5)$, (ordenar las 10 piezas principales, luego peones negros y luego blancos), pero esto deja muchas jugadas que son ilegales o que simplemente no tienen sentido, además de no contar las posiciones con capturas.

- **Complejidad del árbol de juego:** Considerando un factor de ramificación promedio $b \approx 12$ a 15 jugadas legales por posición y una profundidad promedio de partida $d \approx 20$ plies (medios movimientos),

(TODO:por ahora estos 3 numeros son placeholders. son estimaciones sensatas pero estaría bien sacar datos exactos de las estadísticas de generación de partidas)

la complejidad de Shannon b^d para Gardner es:

$$N_{upper} \approx 15^{20} \approx 3,3 \times 10^{23}$$

$$N_{lower} \approx 12^{20} \approx 3,8 \times 10^{21}$$

Estos valores son infinitamente menores que el límite superior del ajedrez estándar (10^{120}), lo que permite que el *self-play* converja con órdenes de magnitud menos recursos computacionales.

Dada la complejidad del entorno, nos gustaría poder modelarlo de modo que el aprendizaje se realice únicamente sobre el estado actual, ya que comprobar el historial de jugadas completo tendría un coste computacional excesivo. Buscamos que la probabilidad de alcanzar un estado futuro dependa únicamente del estado presente, y no del historial de estados anteriores. Esta es la propiedad de Markov.

El ajedrez como Proceso de Decisión de Markov (MDP)

Un proceso estocástico cumple la propiedad de Markov si la probabilidad de alcanzar un estado futuro, s_{t+1} , depende únicamente del estado presente s_t , siendo condicionalmente independiente del historial de estados anteriores. Formalmente:

$$P(s_{t+1}|s_t, s_{t-1}, \dots, s_0) = P(s_{t+1}|s_t)$$

Si definimos el estado del ajedrez únicamente por la disposición física de las piezas en el tablero temos que $s_t = B_t$, entonces el juego no es estrictamente un MDP. Recordemos las reglas mencionadas anteriormente: la triple repetición y la regla de los 50 movimientos tienen en cuenta el historial de estados, no solo el actual.

Para modelar el ajedrez 5×5 como un MDP, es necesario expandir el espacio de estados. Definimos el nuevo estado expandido $s \in \mathcal{S}$ como una tupla $s = (B, p, h)$ donde:

1. B : La disposición exacta de todas las piezas en el tablero de 5×5 en el instante t .
2. p : Un reloj o contador (*half-move clock*) que registra el número de medios movimientos realizados desde la última captura o avance de peón.
3. h : Un registro de las posiciones previas ocurridas desde la última transición irreversible (o, de forma computacionalmente más eficiente, un mapa de frecuencias que contabilice cuántas veces se ha visitado la disposición B actual en la partida en curso).

Al encapsular explícitamente los contadores p y h dentro de la representación del estado actual s_t , el modelo no necesita consultar cronológicamente cómo se llegó a dicho estado. Toda la información temporal y causal necesaria para evaluar la legalidad de las jugadas futuras o detectar empates está autocontenida en el instante presente. Bajo esta formulación, se satisface la propiedad de Markov, lo que nos permite simplificar el modelado.

5.1.3. Arquitectura basada en Transformers: Encoder y Self-Attention

A diferencia de los modelos de lenguaje (LLMs) que generan secuencias texto *token a token* utilizando una arquitectura de decodificador con enmascaramiento causal, la evaluación de un tablero de ajedrez requiere una comprensión bidireccional del estado completo en un instante de tiempo / posición. No es necesario enmascarar la atención para evitar observar tokens futuros, ya que por la propiedad de Markov demostrada anteriormente, una posición no depende de estados posteriores ni anteriores.

Por ello, la arquitectura propuesta se fundamenta en un **Transformer Encoder**. Este modelo procesa simultáneamente todas las casillas del tablero, permitiendo que cada pieza calcule su relevancia respecto a todas las demás sin restricciones de enmascaramiento. El núcleo de esta arquitectura es el mecanismo de Auto-Atención (*Self-Attention*)^[15], definido formalmente como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Donde Q (Query), K (Key) y V (Value) son matrices obtenidas mediante proyecciones lineales de las representaciones del tablero. Dada una entrada X ,

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

W^Q, W^K, W^V son matrices de pesos entrenables, y d_k representa la dimensión de las claves (configurada en **[TODO: definir tamaño de d_k]** para esta implementación). Este mecanismo se replica en cada una de las h cabezas de atención, permitiendo capturar distintas relaciones entre las piezas. El resultado se concatena y se proyecta linealmente de vuelta a la dimensión original de la entrada.

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Donde $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, W^O es una matriz de pesos entrenables y h es el número de cabezas de atención.

5.1.4. Modelado del Aprendizaje por Refuerzo

El problema del ajedrez 5×5 se modela a través de una formulación clásica de Proceso de Decisión de Markov (MDP). En cada paso t , el agente observa el estado actual s_t , elige una acción estocástica basada en una política parametrizada $\pi_\theta(a_t|s_t)$ y pasa a un nuevo estado s_{t+1} . La iteración prosigue hasta el final de la trayectoria (fin de la partida), momento en el cual el agente interactúa con el entorno recibiendo una recompensa terminal (victoria, derrota o empate).

A diferencia de enfoques como MCTS que optimizan una pérdida de entropía cruzada mediante búsqueda explícita, este trabajo emplea una red neuronal de política y valor conjunta sin simulación del futuro. La red se optimiza directamente utilizando la *clipped surrogate objective function* de Proximal Policy Optimization^[16]:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right]$$

donde $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ es el cociente de probabilidades, $\hat{A}_t$ es la estimación de la ventaja y ϵ es el hiperparámetro de recorte (clipping). La red también minimiza el error cuadrático medio en la estimación del valor $V_\theta(s_t)$. La función de pérdida conjunta total es:

$$\mathcal{L} = -L^{CLIP} + c_1 L^{VF} - c_2 L^{ENT}$$

Diseño de la Función de Recompensa

El Aprendizaje por Refuerzo es altamente susceptible al fenómeno de *reward hacking*, donde el agente aprende a explotar lagunas en la función de recompensa para maximizar su puntuación sin alcanzar el objetivo real del problema. Por ejemplo, si se otorgara una recompensa positiva por cada captura de material, el agente podría aprender a retrasar el jaque mate indefinidamente para “farmear” piezas del oponente.

Para evitar este comportamiento subóptimo, se define una función de recompensa dispersa (*sparse reward*) R_t , que únicamente emite una señal al alcanzar un estado terminal final, denominado T :

$$R_t = \begin{cases} 1 & \text{si el estado } T \text{ es victoria} \\ -1 & \text{si el estado } T \text{ es derrota} \\ 0 & \text{si el estado } T \text{ es empate o no terminal} \end{cases}$$

Esta escasez de señal dificulta las fases iniciales del entrenamiento, justificando así la necesidad de técnicas avanzadas o inicialización supervisada para guiar al modelo en las etapas tempranas para acelerar la convergencia de PPO.

5.2. Marco práctico

5.2.1. Codificación del Espacio de Estados (Input)

Para evaluar la Hipótesis 1, se instrumentarán y compararán distintas representaciones del tablero de ajedrez 5x5, analizando su impacto en la asimilación espacial del modelo Transformer.

- **Representación plana simple (Byte Array):** El tablero se representa como un vector unidimensional de 27 *bytes* (índices 0-24), correspondientes a las 25 casillas del tablero de Minichess. Cada casilla almacena un número entero que identifica unívocamente una de las 12 piezas posibles (6 blancas y 6 negras) o el estado de casilla vacía. Adicionalmente se almacenan el número de repeticiones y el número de jugadas sin capturas / jaques en los 2 bytes finales. Esta representación es minimalista pero computacionalmente eficiente, y delega en el Transformer la responsabilidad de deducir la geometría 2D del tablero.
- **Representación espacial (estilo AlphaZero):** El tablero se expande en un tensor multicanal de dimensiones $C \times 5 \times 5$. Cada canal C está asignado a un tipo de pieza por color y actúa como un plano binario indicando su presencia o ausencia. Esta codificación facilita el reconocimiento de patrones espaciales a través de sesgos inductivos explícitos. Se añaden, además, canales binarios adicionales para variables de estado globales como el jugador al que le toca mover y los límites del half-move clock.

- **Representaciones aumentadas (Heurísticas):** Se incorporarán planos adicionales que inyecten información posicional específica para guiar al Transformer, siguiendo las ideas de Czech Et al.^[17] Estos planos binarios codificarán información como: casillas ocupadas por piezas propias y del oponente, posiciones de todas las piezas dando jaque, la topología binaria del tablero (casillas blancas vs. negras), diferencia de material...

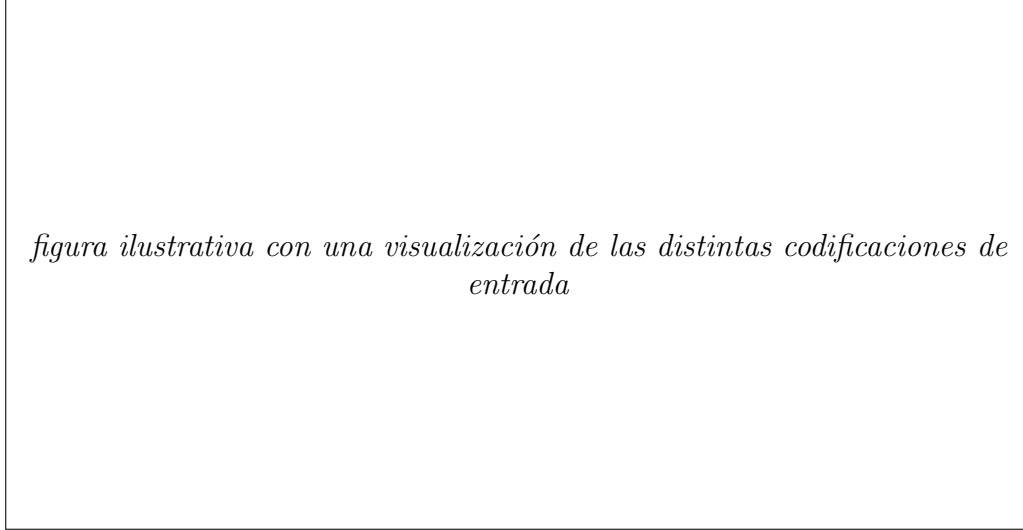


Figura 2: Visualización de codificaciones de entrada. De izquierda a derecha: a, b, c...

5.2.2. Codificación del Espacio de Acciones (Output)

De manera análoga, la forma en que la red proyecta sus predicciones altera drásticamente la topología del espacio de optimización. Se implementarán las siguientes cabezas de salida:

- **Política Plana Discreta (Flat Policy):** Consiste en un vector de salida lineal que mapea todas las posibles transiciones legales del juego. En un tablero 5x5, considerando que una pieza no puede moverse a su propia casilla origen, existen $5^2 \times (5^2 - 1) = 600$ movimientos posicionales puros. Adicionalmente, las promociones de peón requieren un modelado independiente.

En Minichess, los peones en las columnas exteriores (a, e) tienen 2 opciones al coronar (avance o captura simple), mientras que los peones en columnas centrales (b, c, d) tienen 3. Esto resulta en $(2 + 3 + 3 + 3 + 2) = 13$ movimientos de promoción. Multiplicado por las 4 piezas posibles de promoción (Reina, Torre, Alfil, Caballo) y los 2 colores, se añaden 104 neuronas. La salida de la política (*policy head*) será, por tanto, un vector estricto de **704 neuronas**.

- **Política Factorizada (Origen-Destino):** En lugar de predecir sobre el espacio completo de 704 acciones, la salida de la política se bifurca en dos cabezas: una cabeza que predice la casilla de origen del movimiento (vector de tamaño 25) y una segunda cabeza que predice la casilla de destino del movimiento (vector de tamaño 25). Esto reduce el espacio de salida de 704 a 50 dimensiones en total, simplificando la tarea de optimización del gradiente. Se debe añadir también una cabeza adicional para predecir la pieza de promoción, codificada como 9 neuronas (4 piezas posibles por cada color + 1 para no promoción). El total de neuronas asciende a 59.

- **Política con codificación por filas/columnas:** En esta variante, la política se factoriza aún más para predecir por separado la fila y la columna de origen y destino. Esto se implementa mediante cuatro cabezas de salida: una para la fila de origen (5 neuronas), una para la columna de origen (5 neuronas), una para la fila de destino (5 neuronas), una para la columna de destino (5 neuronas) y una para la coronación (9 neuronas). Esto reduce el espacio de salida a solo 29 dimensiones, aunque introduce una mayor complejidad en la decodificación de las acciones.

El objetivo es que, al permitir que el modelo aprenda de forma independiente las dinámicas de filas y columnas y la dinámica de origen y destino, se facilite la generalización al permitir a la red hacer predicciones "parciales".^{al inicio (reconocer la columna de origen sin necesidad de acertar la fila, o viceversa).}

- **Cabeza de valor:** Se incluye una cabeza de valor que predice el valor esperado de la posición actual, lo que permite al modelo aprender a evaluar la calidad de las posiciones además de predecir acciones. Esta cabeza se implementa como una proyección lineal seguida de una activación tanh para acotar la salida en el rango $[-1, 1]$, donde -1 representa una derrota segura, 0 un empate y +1 una victoria segura.

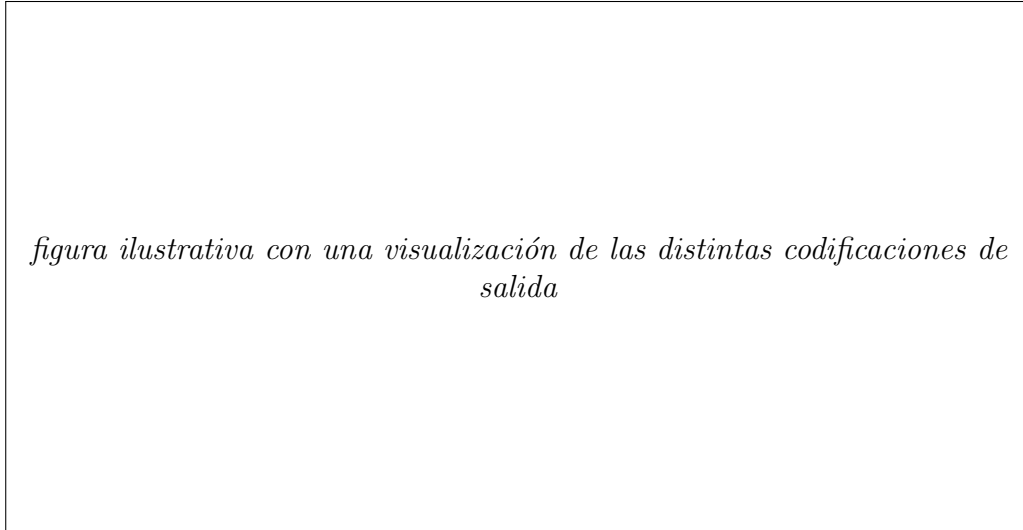


Figura 3: Visualización de codificaciones de salida. De izquierda a derecha: a, b, c...

5.2.3. Arquitectura Transformer

La arquitectura neural propuesta se basa en un modelo **Transformer Encoder** que procesa el tablero de forma bidireccional, permitiendo que cada casilla preste atención a todas las demás. El flujo de procesamiento de la red se describe a continuación:

1. **Capa de Proyección de Entrada:** La capa inicial busca aprender los embeddings que el modelo utilizará. Si la entrada es la representación binaria multicanal $C \times 5 \times 5$, se utiliza una capa convolucional de tamaño de filtro 1×1 o una proyección lineal (**TODO: decidir entre estas 2**) para mapear cada una de las 25 casillas a un vector de dimensión oculta d_{model} (**TODO: decidir tamaño embeddings**).
2. **Codificación Posicional Spatial 2D:** Para inyectar información geométrica y espacial en las capas de atención (ya que los Transformers puros son invariantes ante

permutaciones) (TODO:elegir entre estas 2 en funcion de la dificultad de implementación)...

- se suma un vector de codificación posicional 2D (con componentes seno y coseno correspondientes a las coordenadas de fila y columna de cada casilla) al embedding de cada casilla. (referencia implementación)
- se aplica RoPE (*Rotary Positional Encoding*), el estándar actual de codificación posicional en modelos de lenguaje y Transformers^[18]. Este codifica la posición de cada casilla mediante rotaciones en el espacio de embeddings, permitiendo que el modelo capture relaciones espaciales de manera más flexible. (referencia pytorch)

3. **Pila de Bloques Encoder:** El tensor resultante de dimensiones $25 \times d_{\text{model}}$ se procesa a través de una serie de L capas de codificación. Cada capa contiene:

- Un mecanismo de (*Multi-Head Self-Attention*) sin *causal-masking* como el descrito en la sección 5.1.3. El número de cabezas de atenciones el hiperparámetro h , que será ajustado en base a búsqueda de hiperparámetros.
- Conexiones residuales de inicio a fin y el estándar moderno de normalización, que utiliza *pre-norm* y *RMSNorm*. La convención *pre-norm* aplica normalización antes de cada subcapa, de la forma $z = x + \text{sublayer}(\text{normalization}(x))$, donde *sublayer* es una de las L capas encoder. Se ha demostrado que esta mejora la estabilidad del entrenamiento^[19].

La normalización RMS (*Root Mean Square Layer Normalization*) es utilizada por transformers modernos debido a que reduce la complejidad computacional al no requerir el cálculo de la media para restar al vector original, manteniendo la capacidad de aprendizaje intacta.^[20]

- Una capa *Feed-Forward Network* (FFN), compuesta por dos proyecciones lineales de expansión y contracción y una activación no lineal:

$$\text{FFN}(x) := FC_{\text{expand}}(x) \rightarrow \text{GELU} \rightarrow FC_{\text{project}}(x)$$

4. **Cabezas de Salida:** Tras la última capa del codificador, el estado procesado se bifurca en dos cabezas:

- **Cabeza de Política (*Policy Head*):** Una proyección lineal que reduce el espacio de representación a la dimensión de la política (704 salidas discretas o 50 en la versión factorizada), seguida de una capa Softmax para producir una distribución de probabilidad sobre las acciones legales del estado.
- **Cabeza de Valor (*Value Head*):** Se aplica una capa de agregación global (*pooling*) sobre las casillas, seguida de una proyección lineal a una capa oculta, normalización, y finalmente una proyección escalar con una activación tangente hiperbólica (*tanh*) que acota la salida en el intervalo $[-1, 1]$.

La figura 4 ilustra la arquitectura propuesta, mostrando el flujo de datos desde la entrada de la representación hasta las salidas de las cabezas de política y valor.

Bidirectional Encoder Minichess Transformer (BEMT)

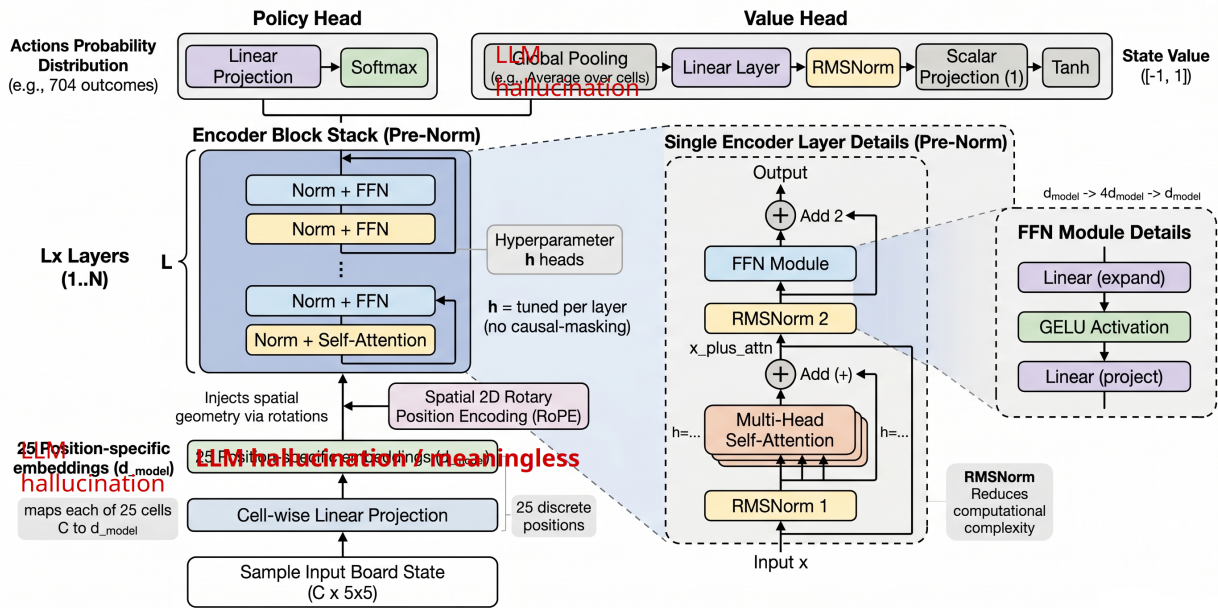


Figura 4: [Placeholder] Diagrama de la arquitectura utilizada (generado con IA, tiene errores / inconsistencias).

5.2.4. Algoritmo de Aprendizaje por Refuerzo: Proximal Policy Optimization (PPO)

Cabe preguntarse: ¿por qué este algoritmo en lugar del MCTS usado en enfoques como AlphaGO o AlphaZero? Clásicamente, metodologías como MCTS y *Expert Iteration* actúan más como planificadores a futuro que como RL propiamente dicho: descubren futuros hipotéticos mediante búsqueda por fuerza bruta al contar con un modelo perfecto del entorno. En ellos, la red neuronal actúa como una memoria supervisada que aproxima las probabilidades de visita ya descubiertas por la búsqueda. Estos sistemas no están aprendiendo *per se* (algo que se obtiene de la experiencia, y por tanto del pasado), sino que están planificando (algo que se obtiene de la previsión del futuro, y por tanto de la simulación, el futuro).

Por el contrario, Proximal Policy Optimization (PPO) es un algoritmo de aprendizaje por refuerzo puro (*Model-Free*). La red neuronal carece de un mecanismo explícito de previsión del futuro mediante un árbol con estados; es ella, iterativamente a través métodos Actor-Crítico y puramente por descenso de gradiente, la que asimila las estrategias óptimas guiada por ensayo y error. Además, al requerirse exactamente una única inferencia por tablero en cada estado, los tableros / estados pueden agruparse en (*batches*) mayores, maximizando la ocupación de GPU.

Mecánica de Optimización y Estimación de Ventaja

El algoritmo PPO actualiza los pesos para evitar colapsar debido a cambios demasiado severos de la política por una actualización errónea. Esta es la función fundamental del hiperparámetro de recorte ϵ (*clipping mechanism*). Este asegura que la actualización no destruya la política existente, al penalizar las actualizaciones estocásticas grandes que diverjan significativamente de la política anterior empleada para recolectar el episodio en cuestión.

Para balancear correctamente el sesgo y la varianza a la hora de determinar lo buena que es la acción tomada respecto a la acción promedio en el estado actual, se emplea la Estimación Generalizada de Ventaja (GAE - *Generalized Advantage Estimation*)^[16]. El algoritmo se puede sintetizar en el pseudocódigo del Algoritmo 1.

TODO revisar comparar con https://spinningup.openai.com/en/latest/algorithms/ppo.html#spinup.ppo_pytorch y [paper original](#)

Algorithm 1 Proximal Policy Optimization (PPO-Clip)

```

1: Inicializar parámetros de política y valor  $\theta$ 
2: for iteración  $i = 1, 2, \dots$  do
3:   Ejecutar política parametrizada  $\pi_{\theta_{old}}$  en  $N$  entornos paralelos durante  $T$  pasos temporales
4:   Calcular las estimaciones de ventaja  $\hat{A}_t$  mediante GAE, en base al valor de  $V_\theta(s_t)$  y las recompensas obtenidas
5:   for época  $k = 1, \dots, K$  do
6:     for cada mini-batch do
7:       Calcular el cociente  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 
8:       Calcular pérdida recortada:  $L^{CLIP} \leftarrow \min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$ 
9:       Calcular pérdida de valor:  $L^{VF} \leftarrow (v_\theta(s_t) - V_t^{target})^2$ 
10:      Calcular pérdida de entropía:  $L^{ENT} \leftarrow H(\pi_\theta(s_t))$ 
11:      Retropropagar para minimizar  $L^{CLIP} + c_1 L^{VF} - c_2 L^{ENT}$ 
12:     end for
13:   end for
14:    $\theta_{old} \leftarrow \theta$ 
15: end for
    
```

En este pseudocódigo, se deben denotar varios aspectos:

- Tanto V_θ como π_θ están parametrizadas por el mismo θ , dado que ambas funciones comparten el backbone de la arquitectura, pero cada una se predice en una de sus cabezas.
- c_1 y c_2 son coeficientes de ponderación para la pérdida de valor y la pérdida de entropía, respectivamente. La función de entropía $H(\pi_\theta(s_t))$ se incluye para fomentar la exploración durante el entrenamiento, y el valor objetivo V_t^{target} se calcula utilizando las recompensas obtenidas y las estimaciones de valor futuro.
- La ventaja en el pseudocódigo anterior se calcula utilizando GAE, y se controla mediante el hiperparámetro λ que ajusta el sesgo-varianza de la estimación. Un valor de λ cercano a 1 favorece una estimación de ventaja con bajo sesgo pero alta varianza (aproxima Monte Carlo), mientras que un valor cercano a 0 produce una estimación con alto sesgo pero baja varianza que aproxima *TD-learning*.

Este hiperparámetro no está presente explícitamente en el pseudocódigo, pero se utiliza en la función de cálculo de $\hat{A}_t$ para determinar cómo se combinan las recompensas futuras y las estimaciones de valor para obtener una ventaja más estable y eficiente. El cálculo de la ventaja $\hat{A}_t$ mediante GAE se realiza utilizando la siguiente fórmula:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

$$\delta_t = r_t + \gamma V_{\theta}(s_{t+1}) - V_{\theta}(s_t)$$

5.2.5. Herramientas y tecnologías empleadas

El desarrollo práctico e investigativo del proyecto se implementará utilizando las siguientes herramientas de desarrollo de software libre:

- **Lenguaje de programación:** Python 3.12+ por su extenso soporte y bibliotecas nativas de Deep Learning.
- **Framework de Deep Learning:** PyTorch para el diseño de redes neuronales, cálculo automático de gradientes y aceleración en GPU.
- **Motor de Ajedrez:** Se emplea **Fairy-Stockfish**, una versión de Stockfish para variantes personalizadas, a través de la API **pyffish** de Python y a través de su CLI propia para generación de datos para la fase de aprendizaje supervisado. Este motor proporciona: (a) validación robusta y rápida de la legalidad de movimientos y reglas de la variante Gardner 5×5 , y (b) evaluaciones heurísticas adecuadas para inicializar la red mediante aprendizaje supervisado.
- **Seguimiento de experimentos:** **Weights & Biases** (wandb) para la visualización en tiempo real del progreso de entrenamiento, entropía de la política, pérdida multiobjetivo y curvas de convergencia.
- **Reproducibilidad, dependencias y manejo de módulos en Python:** uv para la gestión de entornos virtuales y dependencias, asegurando la portabilidad y reproducibilidad del código (uv permite generar archivos de requisitos específicos y fijar una versión de Python).

6. Planificación y seguimiento

La siguiente planificación es una aproximación orientada a la experimentación rápida. Para minimizar la fricción en el desarrollo de los experimentos y la lógica interna del ajedrez Gardner esta se delega a `pyffish`, y del mismo modo, en la medida de lo posible los demás componentes se delegan también. Esta decisión acelera las fases iniciales del proyecto y permite además obtener datos de buena calidad para el preentrenamiento de los modelos. A continuación se presenta el plan de trabajo, dividido en fases:

Fase 0: Investigación previa y documentación inicial

- Investigación del estado del arte en RL y Transformers aplicados al ajedrez.
- Consolidación de objetivos generales y específicos. Creación del marco teórico y práctico para el desarrollo del TFG.

Fase 1: Preparación del Entorno y Generación de Datos Sintéticos

- Integración de `pyffish` para la configuración de la variante `gardner` y validación de movimientos.
- Desarrollo y ejecución de *scripts* de *self-play* basados en evaluaciones a profundidad controlada de Fairy-Stockfish para construir el *dataset* de supervisado inicial.

Fase 2: Arquitectura y Preentrenamiento (Supervisado)

- Implementación del modelo Transformer Encoder y las distintas capas de adaptación (I/O).
- Ejecución de experimentos de aprendizaje supervisado utilizando los *datasets* generados, evaluando la velocidad de convergencia y capacidad de las distintas representaciones de tablero y acción.

Fase 3: Aprendizaje por Refuerzo (Pipeline de Self-Play)

- Construcción del entorno vectorizado para muestreo paralelo guiado por las predicciones de política y valor del Transformer.
- Implementación o integración del algoritmo Actor-Crítico Proximal Policy Optimization (PPO).

Fase 4: Evaluación y Generación de Métricas

- Ejecución de torneos automatizados entre agentes de control (Stockfish limitado, modelos puramente *tabula rasa* frente a modelos preentrenados).
- Extracción y visualización de las métricas de rendimiento para la validación de las hipótesis planteadas.

Fase 5: Documentación y Conclusiones

- Consolidación de los resultados en el documento de Trabajo de Fin de Grado, análisis crítico de las gráficas resultantes y preparación de la defensa.

7. Principales aportaciones

TODO aquí las gráficas!!!

- **No introducir conceptos nuevos. directo al grano.**
- **Agrupar los resultados por cada una de las 3 hipótesis. Insertar las 1 o 2 gráficas correspondientes a cada bloque.**

7.1. Resultados generales en el entorno de Minichess

qué gráficas / tablas puedo aportar para esta hipótesis? no es exactamente un problema cuantificable. quizás sirve con algo como: *"Como hemos visto en el desarrollo anterior, el problema del Minichess mantiene suficiente complejidad para ser utilizado como entorno de estudio a pesar de su tamaño reducido y su condición de weakly-solved"*

7.2. Efectos de la representación sobre la eficiencia y calidad del aprendizaje

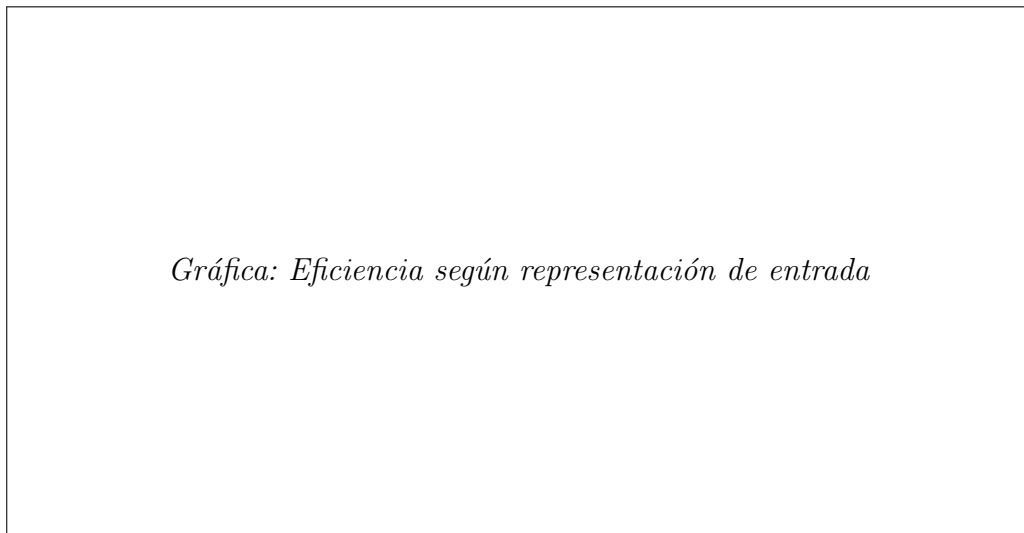


Figura 5: Evolución de la pérdida según las distintas codificaciones del tablero estudiadas.

Representación Input/Output	Iteraciones (PPO)	Winrate VS Heurística
Plana		
Espacial		
Aumentada (Heurísticas)		

Cadro 1: Resultados de rendimiento final comparando representaciones (Hipótesis 2).

7.3. Efectos de la inicialización supervisada sobre la convergencia del aprendizaje por refuerzo

8. Conclusiones

TODO

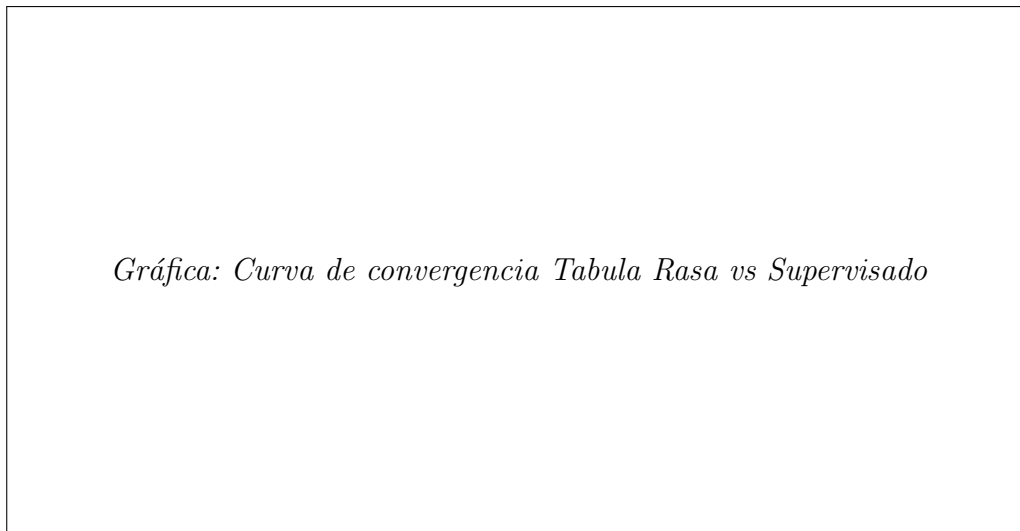


Figura 6: Tasa de victoria y avance del ELO estimado a lo largo del entrenamiento.

Inicialización del Agente	Epochs al pico ELO	ELO final
Tabula rasa		
Pre-entrenado (Supervisado)		

Cadro 2: Comparativa de convergencia pre-entrenamiento vs tabula rasa (Hipótesis 3).

- Análisis crítico de las gráficas, lectura matemática.
- ¿Se confirman las hipótesis o se desmienten?

9. Vías de trabajo futuro

TODO Lo que me quedó en el tintero

TODO limitaciones del enfoque y posibles mejoras

Referencias

- [1] M. R. Segal, “Chess, Chance and Conspiracy,” *Statistical Science*, vol. 22, n.º 1, feb. de 2007, ISSN: 0883-4237. DOI: 10.1214/088342306000000574. URL: <http://dx.doi.org/10.1214/088342306000000574>.
- [2] R. S. Sutton. “The Bitter Lesson.” (2019), URL: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- [3] D. Silver, A. Huang, C. J. Maddison et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, n.º 7587, pp. 484–489, 2016. DOI: 10.1038/nature16961. URL: <https://www.nature.com/articles/nature16961>.
- [4] D. Silver, T. Hubert, J. Schrittwieser et al., “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” *Science*, vol. 362, n.º 6419, pp. 1140–1144, 2018. DOI: 10.1126/science.aar6404. URL: <https://www.science.org/doi/10.1126/science.aar6404>.
- [5] J. Jumper, R. Evans, A. Pritzel et al., “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, n.º 7873, pp. 583–589, 2021. URL: <https://www.nature.com/articles/s41586-021-03819-2>.
- [6] Y. Nasu, “Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi,” en *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 1–7. URL: <https://github.com/asdfjkl/nnue>.
- [7] A. Ruoss, G. Delétang, S. Medapati et al., *Amortized Planning with Large-Scale Transformers: A Case Study on Chess*, 2024. arXiv: 2402.04494 [cs.LG]. URL: <https://arxiv.org/abs/2402.04494>.
- [8] D. Monroe e P. A. Chalmers, *Mastering Chess with a Transformer Model*, 2024. arXiv: 2409.12272 [cs.LG]. URL: <https://arxiv.org/abs/2409.12272>.
- [9] D. Monroe, G. Eilender, P. Chalmers, Z. Tang e A. Anderson, *Chessformer: A Unified Architecture for Chess Modeling*, 2026. arXiv: 2605.19091 [cs.LG]. URL: <https://arxiv.org/abs/2605.19091>.
- [10] E. Jenner, S. Kapur, V. Georgiev, C. Allen, S. Emmons e S. Russell, *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*, 2024. arXiv: 2406.00877 [cs.LG]. URL: <https://arxiv.org/abs/2406.00877>.
- [11] Daniel Monroe. “Transformer Progress in Leela Zero.” (2024), URL: <https://lczero.org/blog/2024/02/transformer-progress/>.
- [12] D. Pritchard e J. Beasley. “Encyclopedia of Chess Variants.” (2007), URL: <https://www.jsbeasley.co.uk/encyc/encyc.pdf>.
- [13] M. Mhalla e F. Prost, “Gardner’s Minichess Variant is solved,” *CoRR*, vol. abs/1307.7118, 2013. arXiv: 1307.7118. URL: <http://arxiv.org/abs/1307.7118>.
- [14] C. E. Shannon, “Programming a Computer for Playing Chess,” *Philosophical Magazine*, vol. 41, n.º 314, pp. 304–317, 1950.
- [15] A. Vaswani, N. Shazeer, N. Parmar et al., “Attention is All you Need,” en *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio et al., eds., vol. 30, Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov, “Proximal Policy Optimization Algorithms,” *CoRR*, vol. abs/1707.06347, 2017. arXiv: 1707.06347. URL: <http://arxiv.org/abs/1707.06347>.
- [17] J. Czech, J. Blüml, K. Kersting e H. Steingrimsson, *Representation Matters for Mastering Chess: Improved Feature Representation in AlphaZero Outperforms Switching to*

- Transformers*, 2024. arXiv: 2304.14918 [cs.AI]. URL: <https://arxiv.org/abs/2304.14918>.
- [18] J. Su, Y. Lu, S. Pan, B. Wen e Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *CoRR*, vol. abs/2104.09864, 2021. arXiv: 2104.09864. URL: <https://arxiv.org/abs/2104.09864>.
- [19] R. Xiong, Y. Yang, D. He et al., “On Layer Normalization in the Transformer Architecture,” *CoRR*, vol. abs/2002.04745, 2020. arXiv: 2002.04745. URL: <https://arxiv.org/abs/2002.04745>.
- [20] A. Gupta, A. Ozdemir e G. Anumanchipalli, *Geometric Interpretation of Layer Normalization and a Comparative Analysis with RMSNorm*, 2025. arXiv: 2409.12951 [cs.LG]. URL: <https://arxiv.org/abs/2409.12951>.

A. Anexos

Cosas a incluir:

- instrucciones reproducibilidad: configuración del entorno, los comandos de consola para lanzar los scripts de entrenamiento y generación de datos...
- pseudocódigos completos
- tablas de hiperparámetros wandb
- partes importantes de código
- explicaciones más en profundidad de cosas que puedan preguntar en la defensa

A.1. Hiperparámetros del Entrenamiento

TODO: rellenar datos tras hyperparam search

Hiperparámetro (Símbolo)	Valor	Descripción
Arquitectura Transformer		
Dimensión oculta (d_{model})		
Número de capas Encoder (L)		
Cabezas de atención (h)		
Aprendizaje por Refuerzo (PPO)		
Tasa de aprendizaje base (lr)		
Batch size (B)		
Número de epochs PPO (K)		
Constante de descuento (γ)		
Parámetro GAE (λ)		
Márgen de Clipping (ϵ)		
Coefficiente de Entropía (c_2)		
Coefficiente de Valor (c_1)		

Cadro 3: Tabla de hiperparámetros de los modelos y del algoritmo PPO.